

Unit 1: Basics: hyperparameter tuning and scaling

Assignment handout

dl-alchemy (after Stanford CS 312: Deep Learning Alchemy)

2026-09-19, draft v0.95

Reconstruction, not the CS 312 handout. Baseline measured; best LR and noise floor follow after calibration.

1 Overview

Read the lecture notes first. This handout scopes out five phenomena around one small transformer. None of the problems is a checklist: each names something to understand and suggests directions, and you decide which experiments would convince you. The unit ends with a 40-minute closed-book quiz in which you predict the outcome of experiments you have not seen, given as diffs against the baseline of [Section 2](#).

How the work is split

You choose experiments, predict, read results and write the report. Claude writes every config and code change, launches and monitors the runs when you say the GPU is free, and returns numbers and plots without interpreting them. To ask for a batch, say in plain English what to run and give a one-line prediction, for example: “LR grid at a quarter of the tokens and at full length; I expect the best LR to be one grid step higher for the short run.” Claude will point out confounders before anything is launched and tell you the GPU hours.

What you will hand in

A short report in your own words ([Section 4](#)). It is not graded. You may cite it from memory in the quiz.

Budget

A baseline run takes about 28 minutes while the GPU is shared with another job (measured: 77k tokens per second) and should take roughly half that on an idle GPU. The hour figures on the problems assume sharing. Doing every problem thoroughly is about 25 to 30 GPU hours; a lean path of about 12 is described in the tip at the end of [Section 3](#). You are not expected to finish everything; you are expected to be able to predict.

2 The baseline

The architecture is the CS 312 example transformer (their **Block** and **Transformer**, reproduced in the course snapshot): pre-norm RMSNorm, attention, MLP, residual connections, learned absolute position embeddings, a final norm and an untied output layer. Every quiz diff is against this model and the recipe below.

Depth 8. As in CS 312.

Width 256, 4 heads of dimension 64, MLP ratio 4. 6.3M parameters in the transformer body, 2.1M in the output layer, 2.2M in the embeddings. CS 312 do not state their width. Measured on this GPU, width 384 would make a Chinchilla-sized run take an hour.

Context 512 tokens. Attention is about 11% of the training FLOPs.

Data FineWeb-Edu, re-tokenized with an 8,192-entry byte-level BPE; 1.78B training tokens on disk.

Validation loss is always measured on the same 8.4M held-out tokens. With GPT-2’s 50k vocabulary the output layer was 55% of the FLOPs and training ran three times slower. Losses are per token of **this** tokenizer and cannot be compared with GPT-2-token losses in papers.

Tokens 120M per run. About 19 tokens per body parameter, close to the Chinchilla ratio. CS 312's baseline uses 614M tokens on a faster GPU.

Batch 64 sequences = 32,768 tokens, about 3,660 steps. Enough steps for schedules to have a shape.

Optimizer AdamW, $\beta_1 = 0.9$, $\beta_2 = 0.95$, $\epsilon = 10^{-8}$, weight decay 0.1, gradient clipping at norm 1. The common LLM recipe, so that published results apply.

Schedule 100 warmup steps, then cosine to 10% of peak over exactly the run length. As in Hoffmann et al.

Peak learning rate the best of $\{10^{-4}, 3 \times 10^{-4}, 10^{-3}, 3 \times 10^{-3}, 10^{-2}, 3 \times 10^{-2}\}$. CS 312's grid and protocol. Calibration reports which value wins for the baseline.

Precision bfloat16 autocast, `torch.compile`.

Knobs you can ask for

Width, depth, heads, MLP ratio, context length; tokens per run, batch size; peak LR, warmup steps, schedule (constant, cosine, linear, WSD with a chosen cooldown length and shape), final LR fraction, the schedule's assumed length; weight decay (coupled to the LR as in PyTorch, or independent), β_1 , β_2 , ϵ , clipping threshold; seed (initialization and data order together, or separately). Anything else is a code diff; describe it and Claude writes it.

What every run records

Training loss per step; validation loss every 250 steps and at the end; gradient norm and clipping rate; parameter and update norms per layer group; tokens per second; whether and when the run diverged. After calibration you also get the **noise floor**: the spread of final validation loss over seeds for the baseline.

3 Problems

Problem (1r-bowl): The shape of the loss-vs-learning-rate curve (about 4 GB10 hrs)

Understand how final validation loss depends on the peak learning rate, and what moves that curve.

- (a) What is the shape? How much does a $3\times$ miss cost on each side, in units of the noise floor? Where is the cliff, and what does a run look like just before it (loss curve, gradient norm, clipping rate)?
- (b) How does the curve move when the run is shorter or longer? When the model is narrower or wider? Decide what to hold fixed and say why.
- (c) Is the best learning rate for the **final** loss also the best at 25% of training? What does that imply for tuning with short runs?

Deliverable: The bowl for the baseline with the noise floor drawn on it, and one plot showing how it moves under the change you chose to study.

Problem (batch): What batch size buys at a fixed token budget (about 5 GB10 hrs)

Hold tokens fixed, vary the batch size over at least a factor of 16, and understand the result.

- (a) With the learning rate re-tuned at each batch size, how does the final loss depend on batch size? Is there a range where it does not matter, and where does it end?

- (b) How does the best learning rate move with batch size? Compare with the linear and square-root rules.
- (c) At the smallest batch sizes, does β_2 matter? Does scaling it as the notes describe change the picture?

Deliverable: Best loss vs batch size, best LR vs batch size, and your estimate of the critical batch size for this model and budget with a sentence on how you would defend it.

Problem (schedule): Warmup and decay (about 4 GB10 hrs)

- (a) How much of the final loss is owed to the decay? Compare a constant learning rate, cosine, and constant plus cooldown, each fairly tuned. Look at whole curves, not only end points.
- (b) What controls the size of the drop during a cooldown: its length, its shape, the peak learning rate, the batch size? The notes' noisy-quadratic picture makes a prediction here. Test it.
- (c) What does warmup do at this scale? Find a setting where it matters and one where it does not.

Deliverable: Loss curves for the schedules on one plot, and a table of final losses with each arm's best LR.

Problem (knobs): Which of the other knobs matter? (about 3 GB10 hrs)

Weight decay, β_2 , ε , the clipping threshold. For each, find out whether a factor-of-ten change moves the final loss by more than the noise floor, and whether it moves the best learning rate. Rank the knobs by how much care they deserve at this scale, and say which you expect to become more important for a larger model or a longer run, and why.

Deliverable: A sensitivity table: knob, range tried, change in loss in units of the noise floor, shift of the best LR.

Problem (scaling): Splitting compute between parameters and tokens (about 6 GB10 hrs, plus 4 for the held-out run)

Take three compute budgets at and below the baseline's, for example $C_0/16$, $C_0/4$ and C_0 . At each, train several model sizes with the token count set by the budget.

- (a) Where is the optimum at each budget, and how flat is the bowl around it? How does the answer depend on whether N and C count the embedding and output matrices?
- (b) Fit a law for the optimal model size and for the loss at the optimum. **Before** running anything at $4C_0$, write down the predicted best size and its loss, with an error bar you believe.
- (c) Then run it. How wrong were you, and which assumption was responsible?
- (d) Does the learning rate need re-tuning across sizes for the fit to be right? What happens to the fitted exponent if you do not?

Deliverable: IsoFLOP curves, the fit, the written prediction with its date, and the held-out result.

Low-Resource Tip: A lean path of about 12 GPU hours

Locate things with quarter-length runs (about 5 minutes each) and confirm only the two or three most informative points at full length. Do `lr-bowl` and `scaling` properly, because every later unit leans on them; pick one of `batch` and `schedule`; do `knobs` only for weight decay. If a week is busy, ask for menu mode: a fixed list of runs, you predict each one, Claude runs them all.

4 The report

Short is fine. It should contain:

1. **Your anchor table.** The noise floor; what a $3\times$ too low and a $3\times$ too high learning rate cost; what halving and doubling the tokens buys; what a $2\times$ mis-sized model costs at fixed compute; the size of the cooldown drop. These numbers are what you will reason from in the quiz, in this unit and later ones.
2. One plot or table per problem you attempted, with one sentence each: what you now believe.
3. Each prediction you made before a batch, and whether it held. Wrong predictions are the valuable ones; say what picture you had and what replaced it.

5 The quiz

Forty minutes, closed book, five to eight questions from easy to hard. Every question is an experiment that was actually run on this machine with this baseline: a description and a diff, then either a ranking of final validation losses (with “=” for differences inside the noise floor) or a number such as a loss difference. Easy questions stay close to the problems above; hard ones combine two knobs or move outside the ranges suggested here. The debrief follows immediately.